

PLC kommunikation og skriv til fil

*Python **snap7** + filskrivning med **open()** og **csv***

Læringsmaal

- ✓ Skrive data til en tekstfil (**.txt**) med **open()** og append mode
- ✓ Tilføje tidsstempler med **datetime** modulet
- ✓ Skrive strukturerede data til en CSV-fil med **csv.writer**
- ✓ Bruge tidsstemplet som filnavn for at organisere logfiler
- ✓ Kombinere PLC-læsning (**snap7**) med fil-logging i en samlet løkke



Indhold

1	Introduktion	2
2	Skriv til tekstfil (.txt)	2
2.1	Skriv en linje til .txt	2
2.2	Skriv tidsstempel til .txt	3
2.3	Tidsstempel som filnavn	3
3	Skriv til CSV-fil	4
3.1	Basal CSV-skrivning	4
3.2	Skriv tidsstempel og værdi til CSV	4
3.3	Tidsstempel som CSV-filnavn	5
4	Kombiner PLC-læsning med fil-logging	5
5	Oversigt over centrale funktioner	7
6	Øvelser	8

1 | Introduktion

I dag kombinerer vi PLC-kommunikation med fil-skrivning. Data der læses fra PLC'en gemmes i en `.txt` eller `.csv` fil med et tidsstempel — en grundlæggende form for **datalogning**.

Hvorfor logge data?

Ved at gemme måledata over tid kan vi:

- Analysere forløb efterfølgende
- Opdage fejl eller ændringer i processen
- Visualisere data i fx Excel eller Matplotlib
- Dokumentere produktion eller testforløb

2 | Skriv til tekstfil (.txt)

Python har en indbygget `open()` funktion til at arbejde med filer. De vigtigste file modes er:

Mode	Navn	Beskrivelse
'r'	Read	Læse mode (fejl hvis filen ikke findes)
'w'	Write	Skrive mode — overskriver eksisterende fil
'a'	Append	Tilføjer til eksisterende fil (opretter hvis ny)

2.1 Skriv en linje til .txt

Kode 1 — Skriv og læs en tekstfil

```
1 # Skriv en linje til filen (append mode)
2 with open('min_fil.txt', 'a') as file:
3     file.write("Dette er en ny linje.\n")
4
5 # Læs og udskriv filens indhold
6 with open('min_fil.txt', 'r') as file:
7     content = file.read()
8     print(content)
```

\n — ny linje

`file.write()` tilføjer **ikke** automatisk linjeskift. Brug `"\n"` for at starte på en ny linje. Med `'a'` (append) beholder filen sit eksisterende indhold — kød koden flere gange og se resultatet.

2.2 Skriv tidsstempel til .txt

Kode 2 — Tidsstempel i tekstfil

```
1 from datetime import datetime
2
3 # Hent og formater aktuelt tidspunkt
4 current_time = datetime.now()
5 formatted_time = current_time.strftime("%Y-%m-%d %H:%M:%S")
6
7 # Skriv tidsstemplet til filen
8 with open('tidsstempel.txt', 'a') as file:
9     file.write(formatted_time + '\n')
10
11 # Laes og udskriv
12 with open('tidsstempel.txt', 'r') as file:
13     print(file.read())
```

2.3 Tidsstempel som filnavn

I stedet for et fast filnavn kan vi bruge tidsstemplet direkte i filnavnet. Det er nyttigt når vi vil gemme separate log-sessioner.

Kode 3 — Tidsstempel som filnavn

```
1 from datetime import datetime
2
3 # Tidsstempel formateret til brug i filnavn (ingen ugyldige tegn)
4 timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
5 filename = f"{timestamp}_log.txt"
6
7 with open(filename, 'a') as file:
8     file.write("Logsession startet.\n")
9     file.write(f"Tidspunkt: {timestamp}\n")
10
11 print(f"Fil oprettet: {filename}")
```

Undgåa ugyldige tegn i filnavne

Tegn som : (kolon) er ugyldige i filnavne paa Windows og macOS. Brug - " i stedet for ":" når du formaterer tidsstemplet til et filnavn, fx "%H-%M-%S" fremfor "%H:%M:%S".

3 | Skriv til CSV-fil

CSV (*Comma Separated Values*) er et simpelt og bredt understøttet format til tabeldata. Python har det indbyggede `csv` modul.

3.1 Basal CSV-skrivning

Kode 4 — Skriv og læs en CSV-fil

```
1 import csv
2
3 # Skriv en række til CSV-filen
4 with open('min_fil.csv', 'a', newline='', encoding='utf-8') as file:
5     writer = csv.writer(file)
6     writer.writerow(["Dette er en ny linje."])
7
8 # Læs og udskriv indholdet
9 with open('min_fil.csv', 'r', encoding='utf-8') as file:
10     reader = csv.reader(file)
11     for row in reader:
12         print(row)
```

Parametre til `open()` ved CSV

<code>newline=""</code>	Forhindr dobbelt linjeskift paa Windows.
<code>encoding='utf-8'</code>	Sikre korrekt håndtering af danske tegn (æ, ø, å).
<code>writer.writerow([])</code>	Accepter en liste — hvert element bliver en kolonne.

3.2 Skriv tidsstempel og værdi til CSV

Kode 5 — Tidsstempel og tilfældig værdi i CSV

```
1 import csv
2 import random
3 from datetime import datetime
4
5 # Formater tidsstempel
6 formatted_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
7 value = random.randint(1, 100)
8
9 # Skriv række til CSV
10 with open('tidsstempel.csv', 'a', newline='', encoding='utf-8') as file:
11     writer = csv.writer(file)
12     writer.writerow([formatted_time, value])
13
14 # Verificer ved at læse tilbage
15 with open('tidsstempel.csv', 'r', encoding='utf-8') as file:
16     reader = csv.reader(file)
17     for row in reader:
18         print(row)
```

3.3 Tidsstempel som CSV-filnavn

Kode 6 — Tidsstempel som CSV-filnavn

```
1 import csv
2 from datetime import datetime
3
4 timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
5 filename = f"{timestamp}_data.csv"
6
7 with open(filename, 'w', newline='', encoding='utf-8') as file:
8     writer = csv.writer(file)
9     writer.writerow(["Tidsstempel", "Vaerdi"])    # Header-raekke
10    writer.writerow([timestamp, 42])
11
12 print(f"CSV-fil oprettet: {filename}")
```

4 | Kombiner PLC-laesning med fil-logging

Nu samler vi det hele: vi laaser et tankniveau fra PLC'en og gemmer det loebende i en CSV-fil med tidsstempel.

 Kode 7 — Log tankniveau fra PLC til CSV

```
1 import csv
2 import time
3 import snap7
4 from snap7.util import get_real
5 from datetime import datetime
6
7 # ===== PLC opsætning =====
8 PLC_IP = "192.168.0.1"
9 RACK = 0
10 SLOT = 1
11 DB_NUMBER = 1
12 TANK_OFFSET = 0
13 TANK_SIZE = 4
14
15 # CSV-fil med tidsstempel som filnavn
16 timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
17 filename = f"{timestamp}_tankniveau.csv"
18
19 try:
20     # ===== PLC forbindelse =====
21     client = snap7.client.Client()
22     client.connect(PLC_IP, RACK, SLOT)
23
24     # Skriv header-raekke til CSV
25     with open(filename, 'w', newline='', encoding='utf-8') as file:
26         writer = csv.writer(file)
27         writer.writerow(["Tidsstempel", "Tankniveau"])
28
29     # ===== Loekke: laes og log =====
30     for _ in range(10):
31         # ===== Laes input =====
32         raw = client.db_read(DB_NUMBER, TANK_OFFSET, TANK_SIZE)
33
34         # ===== Logik =====
35         niveau = round(get_real(raw, 0), 2)
36         ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
37         print(f"{ts} Tankniveau: {niveau}")
38
39         # ===== Skriv output (til fil) ==
40         with open(filename, 'a', newline='', encoding='utf-8') as file:
41             writer = csv.writer(file)
42             writer.writerow([ts, niveau])
43
44         time.sleep(1)
45
46 except snap7.exceptions.Snap7Exception as e:
47     print(f"Snap7 fejl: {e}")
48 except Exception as e:
49     print(f"Uventet fejl: {e}")
50
51 finally:
52     # ===== Disconnect og destroy ==
53     if client.get_connected():
54         client.disconnect()
55         client.destroy()
56     print(f"Logning afsluttet. Data gemt i: {filename}")
```

 Tip — Aaben løkke

Erstat `for _ in range(10):` med `while True:` for en kontinuerlig logning. Brug `Ctrl+C` til at stoppe; haandter `KeyboardInterrupt` som en ekstra `except`-gren for en pæn afslutning.

5 | Oversigt over centrale funktioner

Funktion / mode	Beskrivelse	Modul
<code>open(f, 'a')</code>	Append — tilføjer til filen	<code>indbygget</code>
<code>open(f, 'w')</code>	Write — overskriver filen	<code>indbygget</code>
<code>open(f, 'r')</code>	Read — læs indholdet	<code>indbygget</code>
<code>file.write(s)</code>	Skriv streng til filen	<code>indbygget</code>
<code>file.read()</code>	Laes hele filens indhold	<code>indbygget</code>
<code>csv.writer(f)</code>	Opret CSV-forfatter	<code>csv</code>
<code>writer.writerow([])</code>	Skriv en række til CSV	<code>csv</code>
<code>csv.reader(f)</code>	Laes CSV række for række	<code>csv</code>
<code>datetime.now()</code>	Aktuelt dato og tidspunkt	<code>datetime</code>
<code>.strftime("...")</code>	Formater tidsstempel som streng	<code>datetime</code>
<code>time.sleep(n)</code>	Vent <i>n</i> sekunder	<code>time</code>

6 | Øvelser

☰ Øvelsesopgaver

1. Exercise 1 — Tankniveau til CSV

Læs tankniveauet fra PLC'en og skriv det til en CSV-fil sammen med et tidsstempel. CSV-filen skal have følgende format:

```
Tidsstempel, Tankniveau
2024-06-01 12:00:00, 75
2024-06-01 12:01:00, 80
```

2. Exercise 2 — Flere værdier til CSV

Udvid Exercise 1 til at logge tidsstempel, tankniveau, tankniveau i procent og flowmeter i samme CSV-fil.

3. Exercise 3 — On/off controller med logging

Lav en on/off controller for tankniveauet. Naar tankniveauet naer setpointet skal det svinge med en amplitude paa 10% af setpointet. Under 5% under setpoint fyldes tanken; over setpoint falder det. Log tankniveauet med tidsstempel over en passende periode.

4. Exercise 4 (valgfri) — PI controller

Implementer en PI controller for tankniveauet i Emulate3D og log data til en CSV-fil. En PI controller kombinerer:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau$$

I kode:

```
 PI controller (skelet)
1 error = setpoint - current_level
2 integral = integral + error * dt
3 control = Kp * error + Ki * integral
```

Naar opgaven er faerdig, visualiser resultatet i Excel.